

# **Design and Implementation of a Parameterized Carry Lookahead Adder Using Verilog HDL**

## **Abstract**

This project presents the design, simulation, synthesis, and implementation of a parameterized Carry Lookahead Adder (CLA) using Verilog HDL. The objective of the project was to design a high-speed adder architecture capable of reducing carry propagation delay compared to conventional Ripple Carry Adders (RCA). The design was modularized into Generation-Propagation, Carry Computation, and Sum Computation blocks to improve readability and scalability.

Functional verification was performed using directed and randomized test cases. Simulation waveforms and TCL console outputs confirmed correct operation for multiple input combinations and different parameterized bit-widths. The design was synthesized and implemented successfully using Vivado targeting the Xilinx Zynq-7000 FPGA family. Power analysis and implementation reports were also generated and analyzed.

## **1. Introduction**

Adders are one of the most fundamental arithmetic components used in digital systems, processors, signal processing units, and embedded hardware. Conventional Ripple Carry Adders suffer from propagation delay because each carry bit depends on the previous carry output.

Carry Lookahead Adders (CLA) improve performance by computing carry signals in parallel using Generate and Propagate logic. This significantly reduces delay and improves speed for larger bit-width arithmetic operations.

The objective of this project is to implement a parameterized CLA architecture using Verilog HDL and validate its functionality through simulation and FPGA implementation flow.

## **2. Objectives**

The major objectives of this project are:

- To understand Carry Lookahead Adder architecture
- To implement modular RTL design using Verilog HDL
- To perform functional verification using testbenches
- To analyze timing and implementation reports
- To understand FPGA synthesis and implementation flow in Vivado
- To perform randomized verification testing
- To develop scalable parameterized digital hardware modules

### 3. Theory of Carry Lookahead Adder

In a Ripple Carry Adder, each carry output depends on the previous carry input, resulting in sequential propagation delay.

Carry Lookahead Adders improve speed by predicting carry values using Generate and Propagate signals.

For each bit:

$$G_i = A_i B_i$$

Generate signal indicates whether a carry will definitely be generated.

$$P_i = A_i + B_i$$

Propagate signal indicates whether an incoming carry will propagate forward.

Carry equations are computed as:

$$C_1 = G_0 + P_0 C_{in}$$

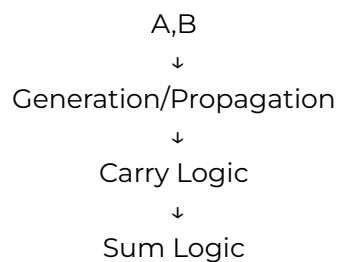
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$

The sum output is calculated using:

$$Sum_i = A_i \oplus B_i \oplus C_{i-1}$$

This parallel carry computation reduces propagation delay and improves overall performance

### 4. Block Diagram



↓  
Sum,Cout

## 5. Design Methodology

The design was implemented using a modular approach.

### Generation and Propagation Module

This module generates:

- Generate signals (G)
- Propagate signals (P)
- using input operands A and B.

Logic used:

- $G = A \& B$
- $P = A | B$

In this implementation, the propagate signal was defined using logical OR operation to simplify generalized carry propagation logic.

The module was parameterized using:

- parameter  $N = 8$

The parameterized architecture enables easy scalability of the CLA design for different bit-width arithmetic operations without modifying the core logic.

### Carry Computation Module

The Carry module computes carry outputs using nested loop logic based on Generate and Propagate signals.

The carry computation was generalized for any value of  $N$  using parameterized loops. This enabled implementation for multiple bit-widths without rewriting the logic.

The module performs parallel carry computation instead of sequential propagation, improving speed compared to Ripple Carry Adders.

### Sum Computation Module

The Sum module computes final output bits using XOR operations between:

- input bits
- carry inputs

The module also generates the final carry-out signal.

### Top Verification Module

The top module performs:

- Directed testing
- Randomized testing
- Functional verification

Verification was performed using:

- \$monitor
- \$random
- \$display

statements.

The following condition was continuously verified:

- $(A + B + C\_1 == \{Cout, sum\})$

A “Match = 1” output indicated successful verification.

## 6. Tools and Technologies Used

| Tool / Technology | Purpose                      |
|-------------------|------------------------------|
| Verilog HDL       | RTL Design                   |
| Xilinx Vivado     | Synthesis and Implementation |
| Vivado Simulator  | Functional Simulation        |
| TCL Console       | Design Flow and Reports      |
| FPGA Device       | xc7z010clg225-1              |

## 7. Simulation and Verification

Functional verification was performed using:

- Directed test vectors
- Randomized test cases

The design was tested for:

- N = 1
- N = 4
- N = 8
- N = 16

The primary implementation was performed for N = 8, while additional simulations for N = 1, 4, and 16 were used to validate parameterized scalability.

Simulation outputs confirmed correct functionality for:

- normal addition
- overflow conditions
- carry propagation
- randomized inputs

All test cases successfully produced:

Match = 1

indicating correct operation.

### **Directed Testing**

Directed test cases included:

- All zeros
- Maximum value addition
- Carry-in testing
- Overflow conditions

These tests verified correct carry generation and sum computation.

### **Randomized Testing**

- Randomized testing was performed using:  
"\$random"  
to generate multiple input combinations automatically.
- The expected result was compared with:  
{Cout, sum}  
to verify correctness.

### **Simulation Environment**

The design was simulated using the Vivado Simulator with a timescale of 1ns/1ps.

Verification was performed by comparing the CLA output against the expected arithmetic result.

## 8. Synthesis and FPGA Implementation

The design was synthesized and implemented successfully in Vivado.

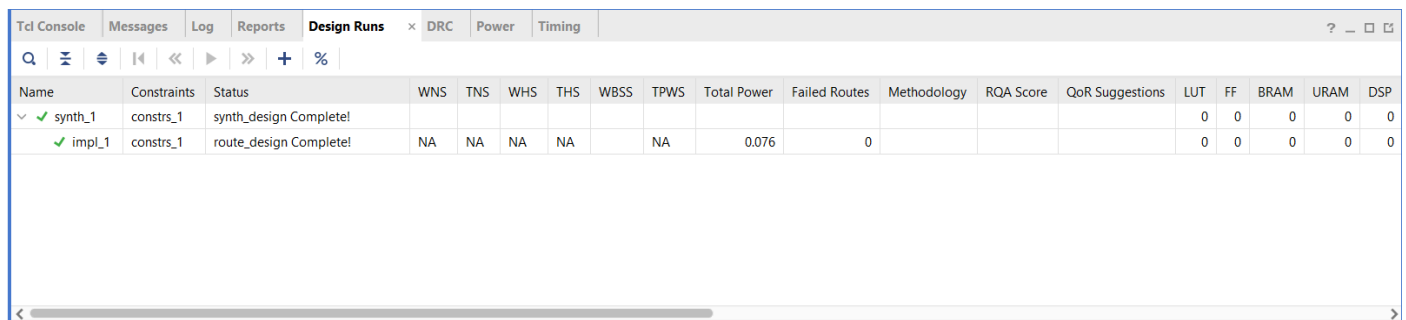
The following stages were completed:

- RTL elaboration
- Synthesis
- Optimization
- Placement
- Routing
- Implemented design generation

The implemented design view confirmed successful routing and resource mapping on the target FPGA device.

Bitstream generation was not completed due to missing FPGA pin constraint assignments (.xdc constraints). However, the implementation flow completed successfully.

The below screenshot confirms the progress:



The screenshot shows the 'Design Runs' tab in the Vivado IDE. It displays a table with columns for Name, Constraints, Status, WNS, TNS, WHS, THS, WBSS, TPWS, Total Power, Failed Routes, Methodology, RQA Score, QoR Suggestions, LUT, FF, BRAM, URAM, and DSP. Two runs are listed: 'synth\_1' and 'impl\_1'. 'synth\_1' is marked as 'synth\_design Complete!' and 'impl\_1' is marked as 'route\_design Complete!'. The 'Total Power' for 'impl\_1' is 0.076 and 'Failed Routes' is 0.

| Name      | Constraints | Status                 | WNS | TNS | WHS | THS | WBSS | TPWS | Total Power | Failed Routes | Methodology | RQA Score | QoR Suggestions | LUT | FF | BRAM | URAM | DSP |
|-----------|-------------|------------------------|-----|-----|-----|-----|------|------|-------------|---------------|-------------|-----------|-----------------|-----|----|------|------|-----|
| ✓ synth_1 | constrs_1   | synth_design Complete! |     |     |     |     |      |      |             |               |             |           |                 | 0   | 0  | 0    | 0    | 0   |
| ✓ impl_1  | constrs_1   | route_design Complete! | NA  | NA  | NA  | NA  |      | NA   | 0.076       | 0             |             |           |                 | 0   | 0  | 0    | 0    | 0   |

## 9. Resource Utilization

The implemented CLA design utilized a small fraction of the available FPGA resources due to its lightweight combinational architecture.

The implementation reports confirmed efficient utilization of:

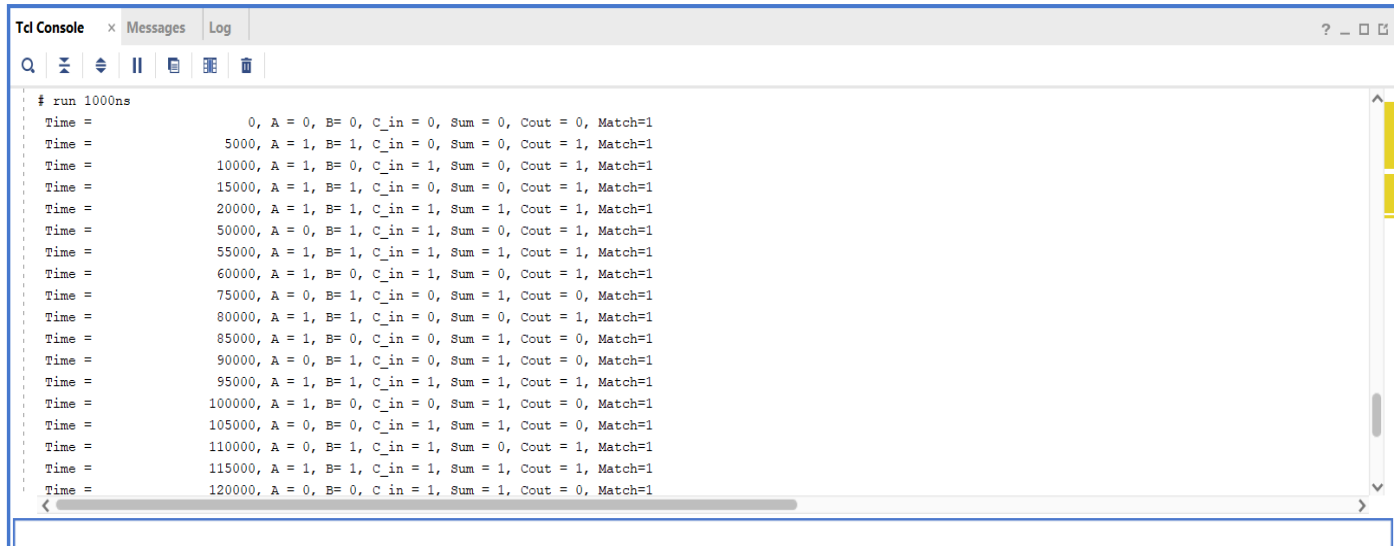
- Lookup Tables (LUTs)
- Routing resources
- Logic elements

The low utilization demonstrates the compactness and scalability of the design.

## 10. Simulation Results

The following outputs were obtained as a log, in TCL console: (Note that the outputs are not till the end)

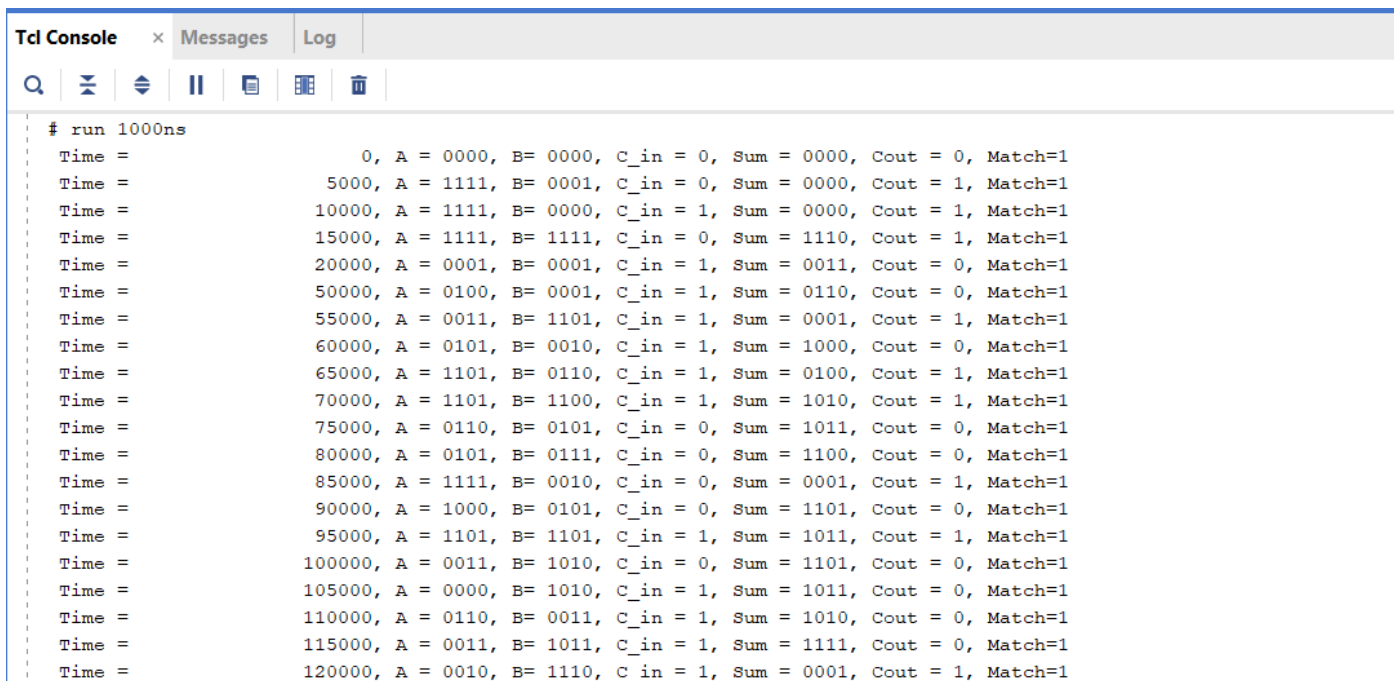
**Figure 1: TCL Console Output for N=1**



The screenshot shows the TCL Console window with the following output:

```
# run 1000ns
Time = 0, A = 0, B= 0, C_in = 0, Sum = 0, Cout = 0, Match=1
Time = 5000, A = 1, B= 1, C_in = 0, Sum = 0, Cout = 1, Match=1
Time = 10000, A = 1, B= 0, C_in = 1, Sum = 0, Cout = 1, Match=1
Time = 15000, A = 1, B= 1, C_in = 0, Sum = 0, Cout = 1, Match=1
Time = 20000, A = 1, B= 1, C_in = 1, Sum = 1, Cout = 1, Match=1
Time = 50000, A = 0, B= 1, C_in = 1, Sum = 0, Cout = 1, Match=1
Time = 55000, A = 1, B= 1, C_in = 1, Sum = 1, Cout = 1, Match=1
Time = 60000, A = 1, B= 0, C_in = 1, Sum = 0, Cout = 1, Match=1
Time = 75000, A = 0, B= 1, C_in = 0, Sum = 1, Cout = 0, Match=1
Time = 80000, A = 1, B= 1, C_in = 0, Sum = 0, Cout = 1, Match=1
Time = 85000, A = 1, B= 0, C_in = 0, Sum = 1, Cout = 0, Match=1
Time = 90000, A = 0, B= 1, C_in = 0, Sum = 1, Cout = 0, Match=1
Time = 95000, A = 1, B= 1, C_in = 1, Sum = 1, Cout = 1, Match=1
Time = 100000, A = 1, B= 0, C_in = 0, Sum = 1, Cout = 0, Match=1
Time = 105000, A = 0, B= 0, C_in = 1, Sum = 1, Cout = 0, Match=1
Time = 110000, A = 0, B= 1, C_in = 1, Sum = 0, Cout = 1, Match=1
Time = 115000, A = 1, B= 1, C_in = 1, Sum = 1, Cout = 1, Match=1
Time = 120000, A = 0, B= 0, C_in = 1, Sum = 1, Cout = 0, Match=1
```

**Figure 2: TCL Console Output for N=4**



The screenshot shows the TCL Console window with the following output:

```
# run 1000ns
Time = 0, A = 0000, B= 0000, C_in = 0, Sum = 0000, Cout = 0, Match=1
Time = 5000, A = 1111, B= 0001, C_in = 0, Sum = 0000, Cout = 1, Match=1
Time = 10000, A = 1111, B= 0000, C_in = 1, Sum = 0000, Cout = 1, Match=1
Time = 15000, A = 1111, B= 1111, C_in = 0, Sum = 1110, Cout = 1, Match=1
Time = 20000, A = 0001, B= 0001, C_in = 1, Sum = 0011, Cout = 0, Match=1
Time = 50000, A = 0100, B= 0001, C_in = 1, Sum = 0110, Cout = 0, Match=1
Time = 55000, A = 0011, B= 1101, C_in = 1, Sum = 0001, Cout = 1, Match=1
Time = 60000, A = 0101, B= 0010, C_in = 1, Sum = 1000, Cout = 0, Match=1
Time = 65000, A = 1101, B= 0110, C_in = 1, Sum = 0100, Cout = 1, Match=1
Time = 70000, A = 1101, B= 1100, C_in = 1, Sum = 1010, Cout = 1, Match=1
Time = 75000, A = 0110, B= 0101, C_in = 0, Sum = 1011, Cout = 0, Match=1
Time = 80000, A = 0101, B= 0111, C_in = 0, Sum = 1100, Cout = 0, Match=1
Time = 85000, A = 1111, B= 0010, C_in = 0, Sum = 0001, Cout = 1, Match=1
Time = 90000, A = 1000, B= 0101, C_in = 0, Sum = 1101, Cout = 0, Match=1
Time = 95000, A = 1101, B= 1101, C_in = 1, Sum = 1011, Cout = 1, Match=1
Time = 100000, A = 0011, B= 1010, C_in = 0, Sum = 1101, Cout = 0, Match=1
Time = 105000, A = 0000, B= 1010, C_in = 1, Sum = 1011, Cout = 0, Match=1
Time = 110000, A = 0110, B= 0011, C_in = 1, Sum = 1010, Cout = 0, Match=1
Time = 115000, A = 0011, B= 1011, C_in = 1, Sum = 1111, Cout = 0, Match=1
Time = 120000, A = 0010, B= 1110, C_in = 1, Sum = 0001, Cout = 1, Match=1
```

**Figure 3: TCL Console Output for N=16**

```
# run 1000ns
Time = 0, A = 0000000000000000, B= 0000000000000000, C_in = 0, Sum = 0000000000000000, Cout = 0, Match=1
Time = 5000, A = 1111111111111111, B= 0000000000000001, C_in = 0, Sum = 0000000000000000, Cout = 1, Match=1
Time = 10000, A = 1111111111111111, B= 0000000000000000, C_in = 1, Sum = 0000000000000000, Cout = 1, Match=1
Time = 15000, A = 1111111111111111, B= 1111111111111111, C_in = 0, Sum = 1111111111111110, Cout = 1, Match=1
Time = 20000, A = 0000000000000001, B= 0000000000000001, C_in = 1, Sum = 0000000000000011, Cout = 0, Match=1
Time = 50000, A = 0011010100100100, B= 0101111010000001, C_in = 1, Sum = 1001001110100110, Cout = 0, Match=1
Time = 55000, A = 0101011001100011, B= 011101100001101, C_in = 1, Sum = 1101000101110001, Cout = 0, Match=1
Time = 60000, A = 1000010001100101, B= 0101001000010010, C_in = 1, Sum = 1101011001110000, Cout = 0, Match=1
Time = 65000, A = 1100110100001101, B= 1111000101110110, C_in = 1, Sum = 1011111010000100, Cout = 1, Match=1
Time = 70000, A = 0101011111101101, B= 1111011110001100, C_in = 1, Sum = 0100111101111010, Cout = 1, Match=1
Time = 75000, A = 0010010011000110, B= 1000010011000101, C_in = 0, Sum = 1010100110001011, Cout = 0, Match=1
Time = 80000, A = 1111011111100101, B= 0111001001110111, C_in = 0, Sum = 0110101001011100, Cout = 1, Match=1
Time = 85000, A = 1101101110001111, B= 0110100111110010, C_in = 0, Sum = 0100010110000001, Cout = 1, Match=1
Time = 90000, A = 0111101011101000, B= 0100111011000101, C_in = 0, Sum = 1100100110101101, Cout = 0, Match=1
Time = 95000, A = 0010100010111101, B= 0101100000101101, C_in = 1, Sum = 1000000011101011, Cout = 0, Match=1
Time = 100000, A = 0110001001100011, B= 1000011100001010, C_in = 0, Sum = 1110100101101101, Cout = 0, Match=1
Time = 105000, A = 0010000100100000, B= 0100010110101010, C_in = 1, Sum = 0110011011001011, Cout = 0, Match=1
Time = 110000, A = 0011111010010110, B= 1011100000010011, C_in = 1, Sum = 1111011010101010, Cout = 0, Match=1
Time = 115000, A = 1101011001010011, B= 1101110101101011, C_in = 1, Sum = 1011001110111111, Cout = 1, Match=1
Time = 120000, A = 0100101000000010, B= 0011111010101110, C_in = 1, Sum = 1000100010110001, Cout = 0, Match=1
```

## 11. Output Waveforms

The simulation waveforms were obtained during behavioral simulation.

The waveforms visually confirmed:

- successful addition
- correct carry out
- Match with expected sum

The waveforms were analysed to verify correct design behavior

**Figure 4: Timing Waveforms for N=1**



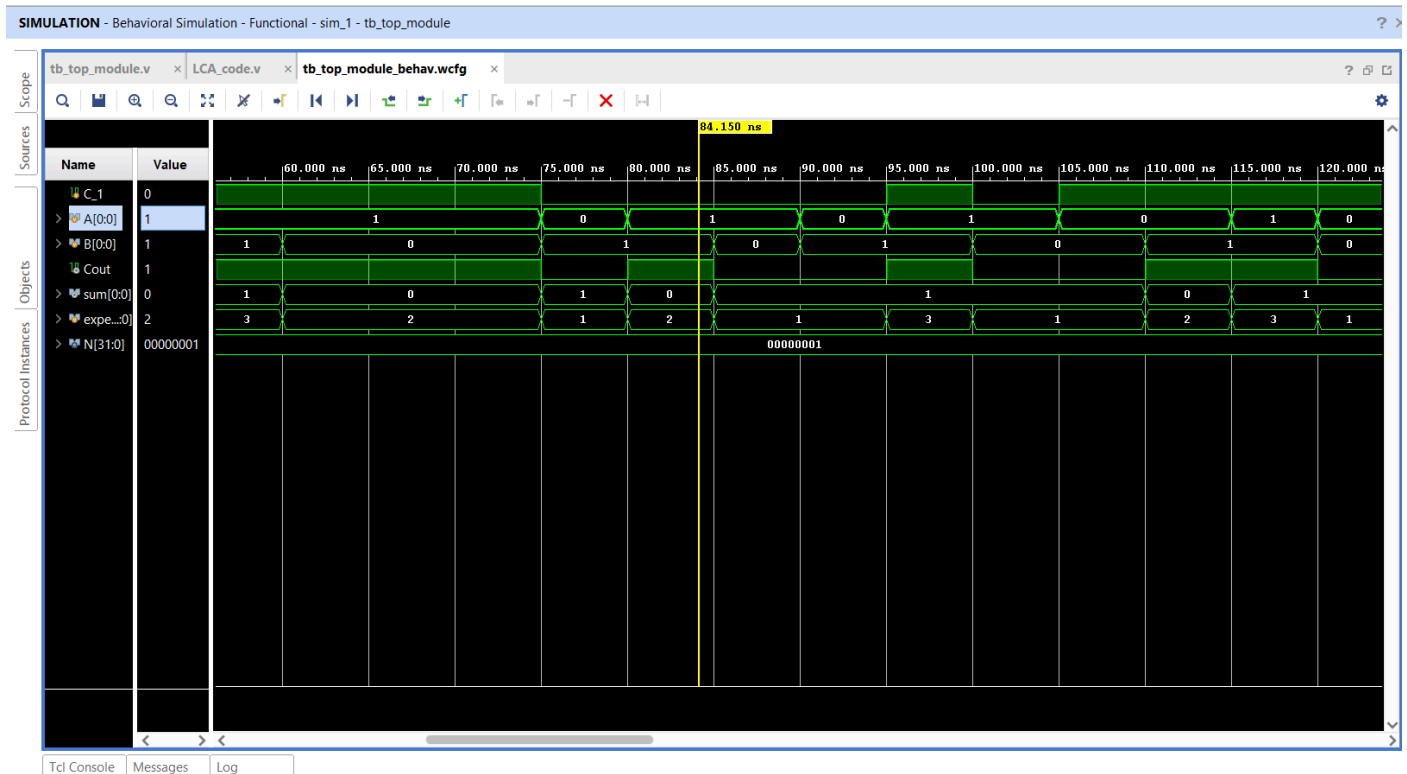
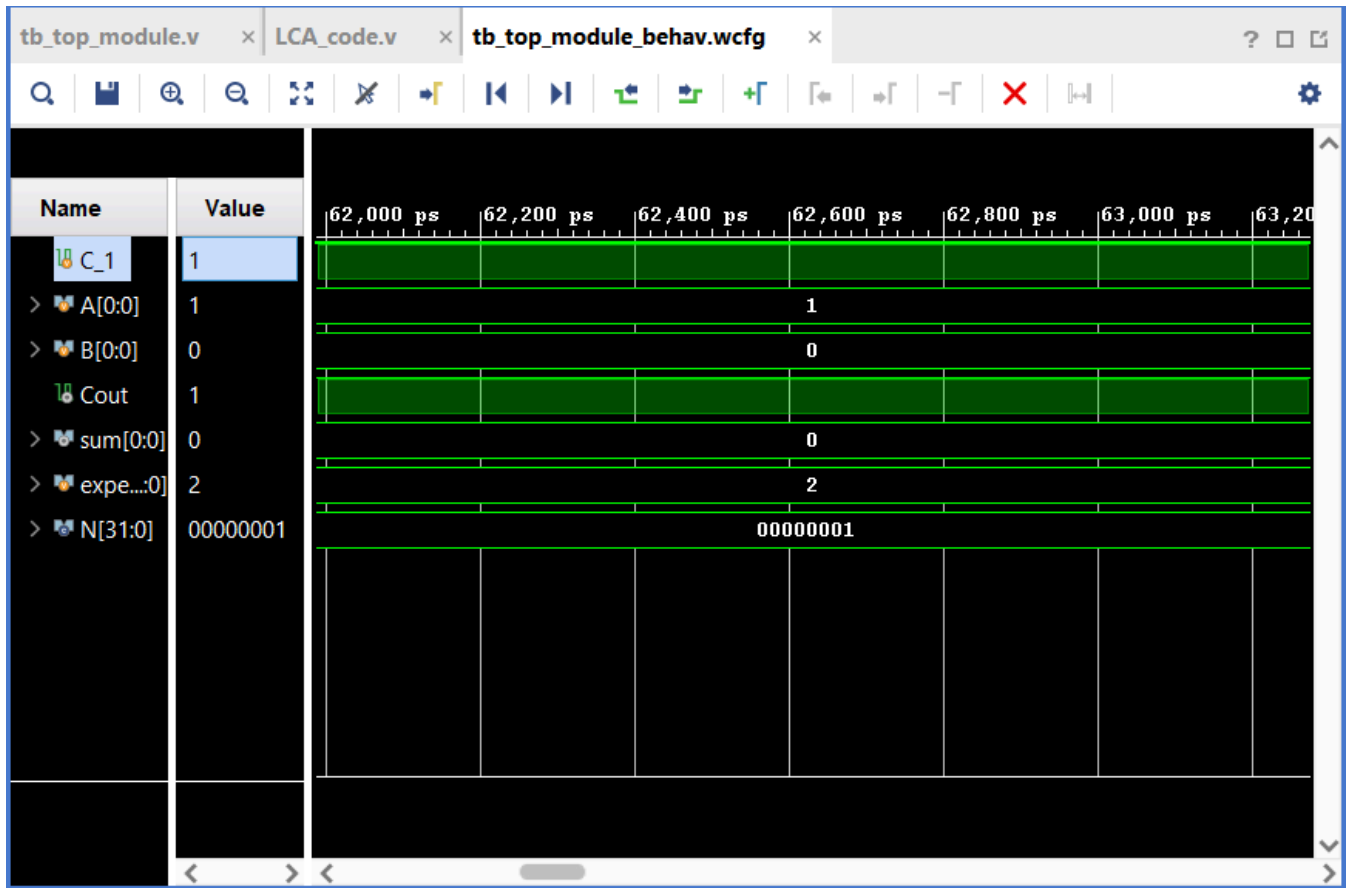


Figure 5: Timing Waveforms for N=4

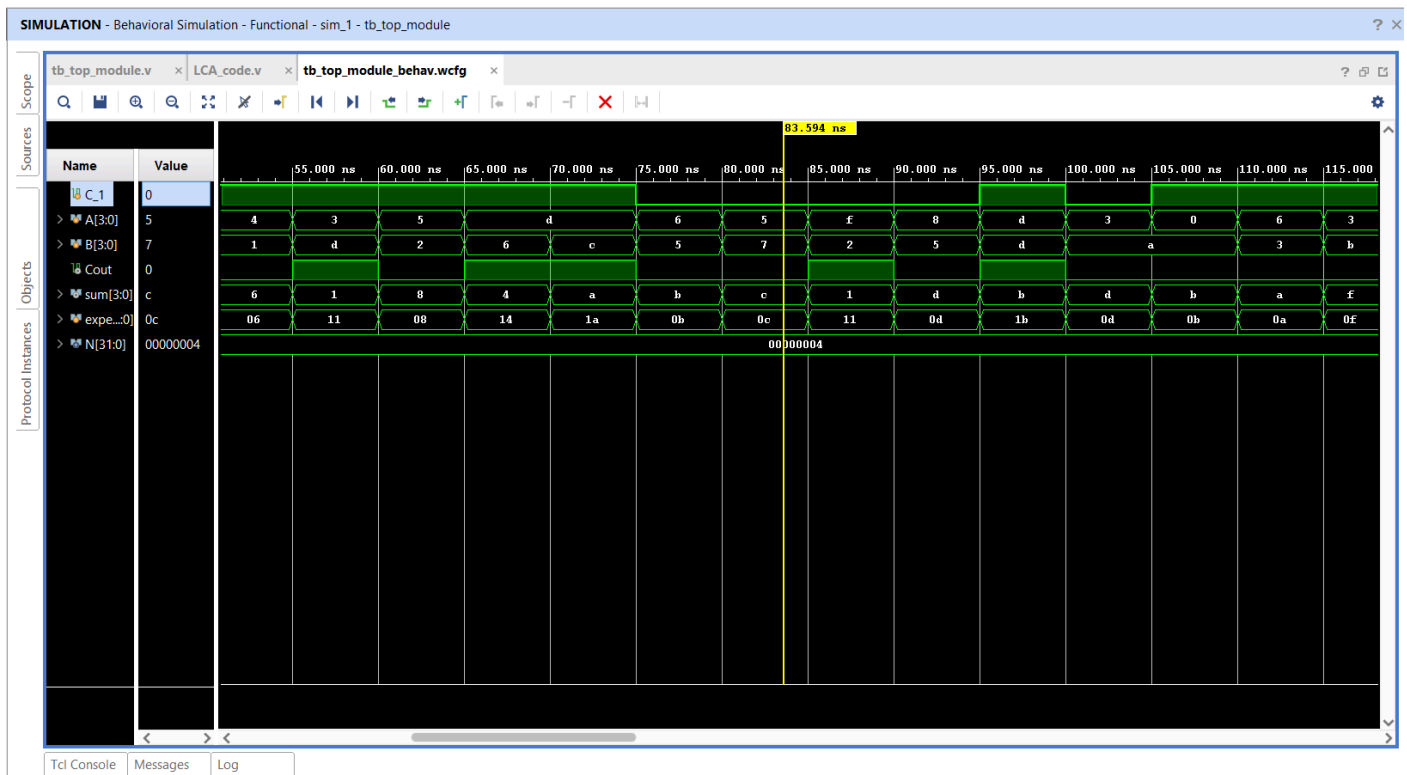
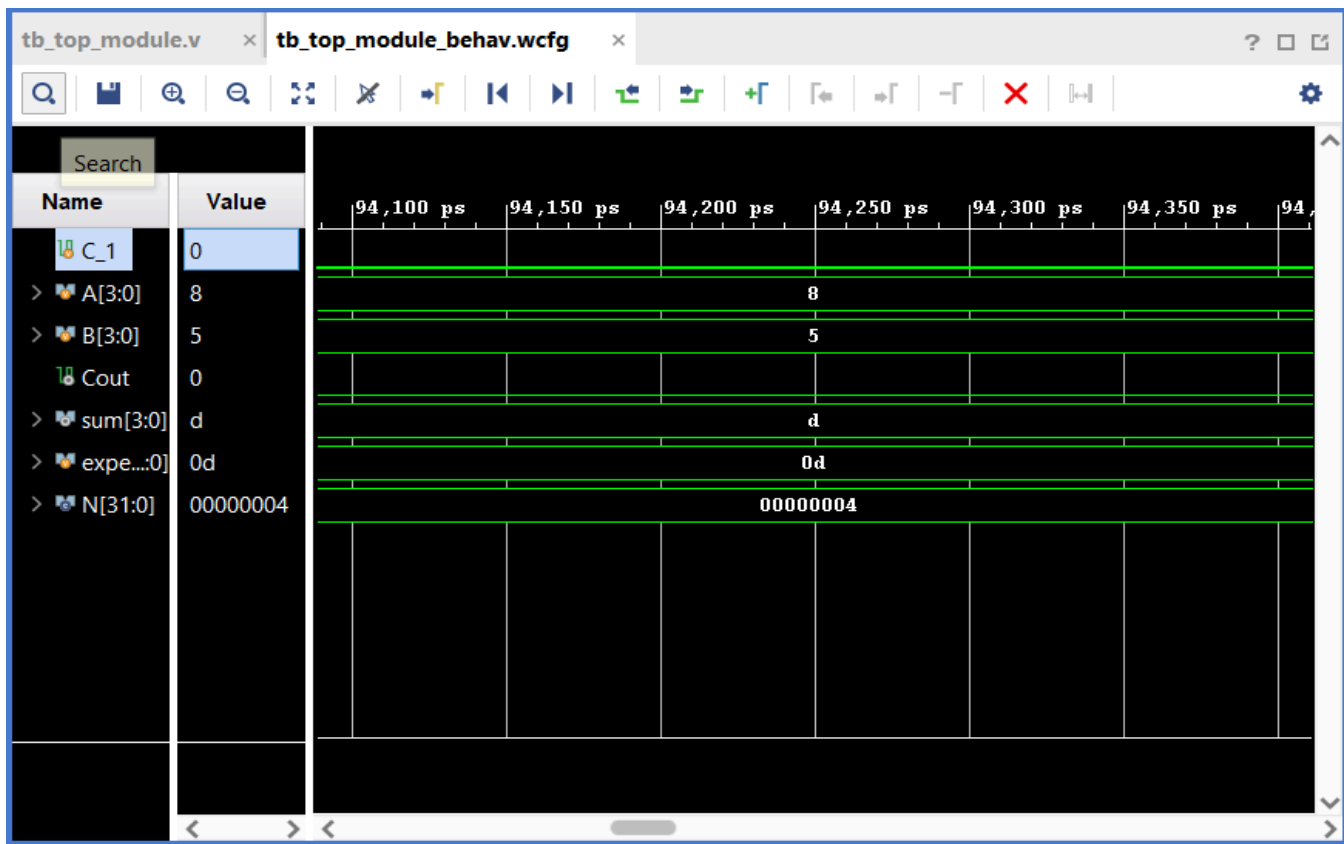
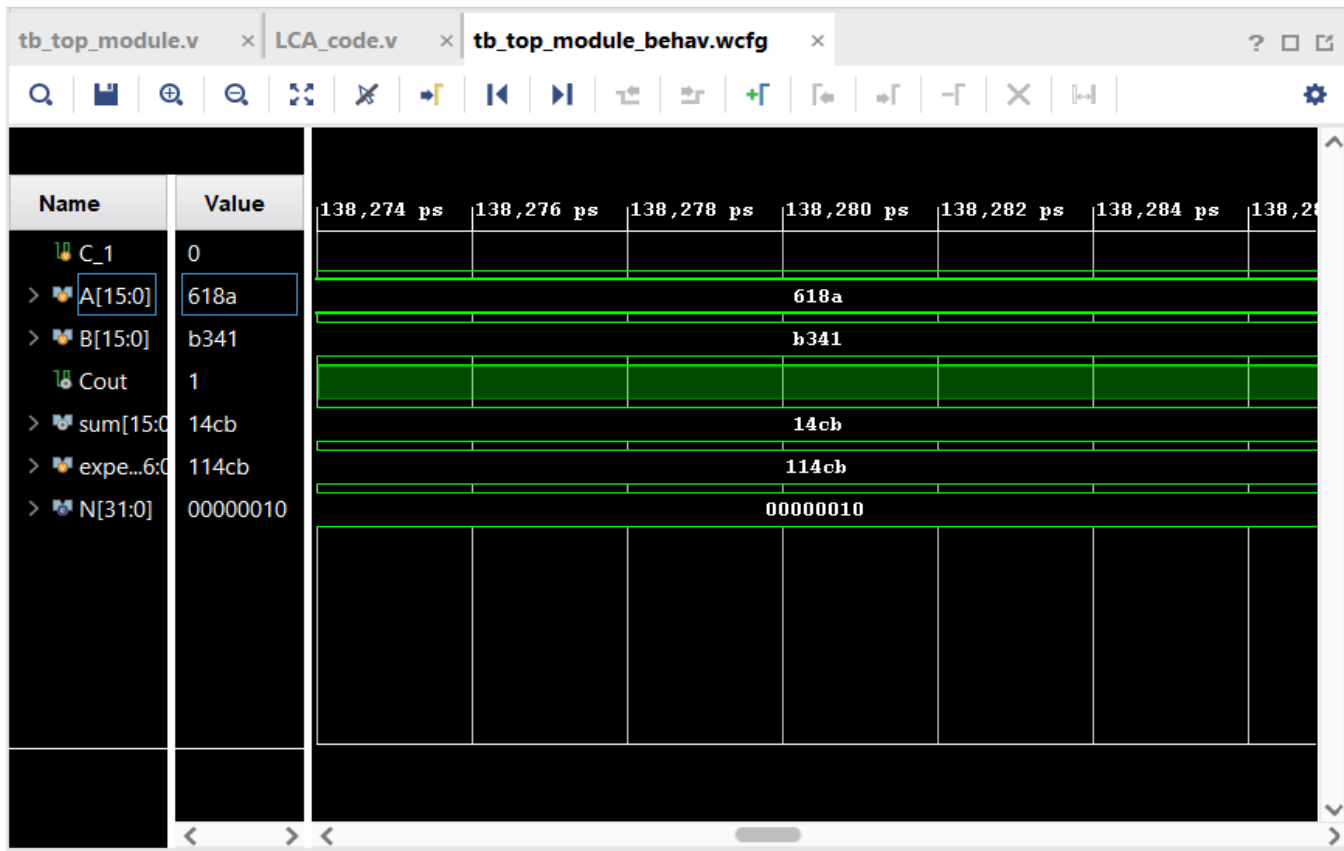


Figure 6: Timing Waveform for N=16



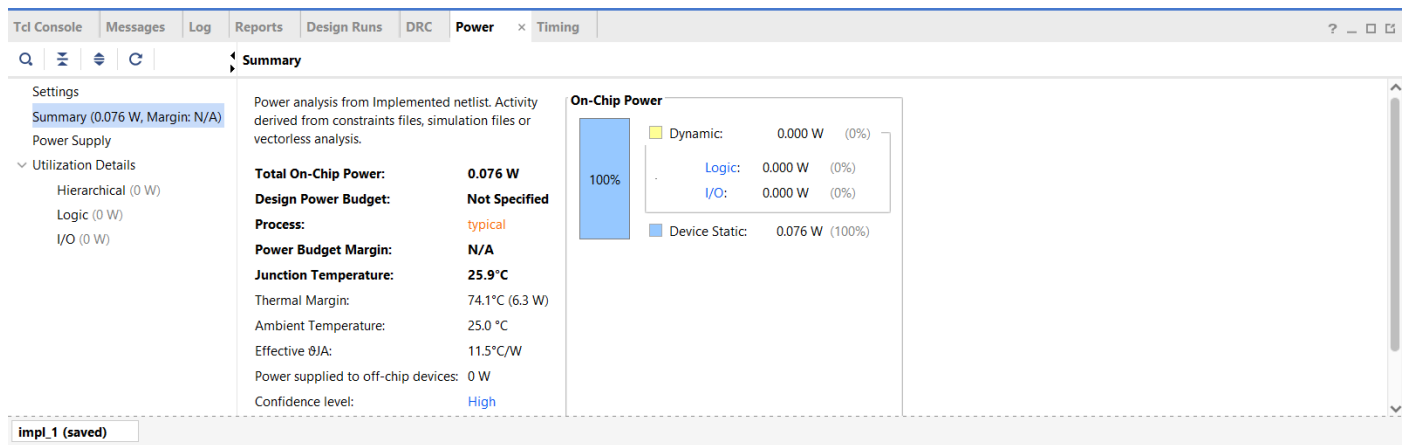
## 12. Power Analysis

Power analysis was performed using Vivado Power Reports.

Observed results:

- Total On-Chip Power  $\approx 0.076$  W
- Confidence Level: High
- Junction Temperature  $\approx 25.9^{\circ}\text{C}$
- The design consumed very low dynamic power because the project primarily consisted of combinational arithmetic logic.

**Figure 7: Power Analysis Report**



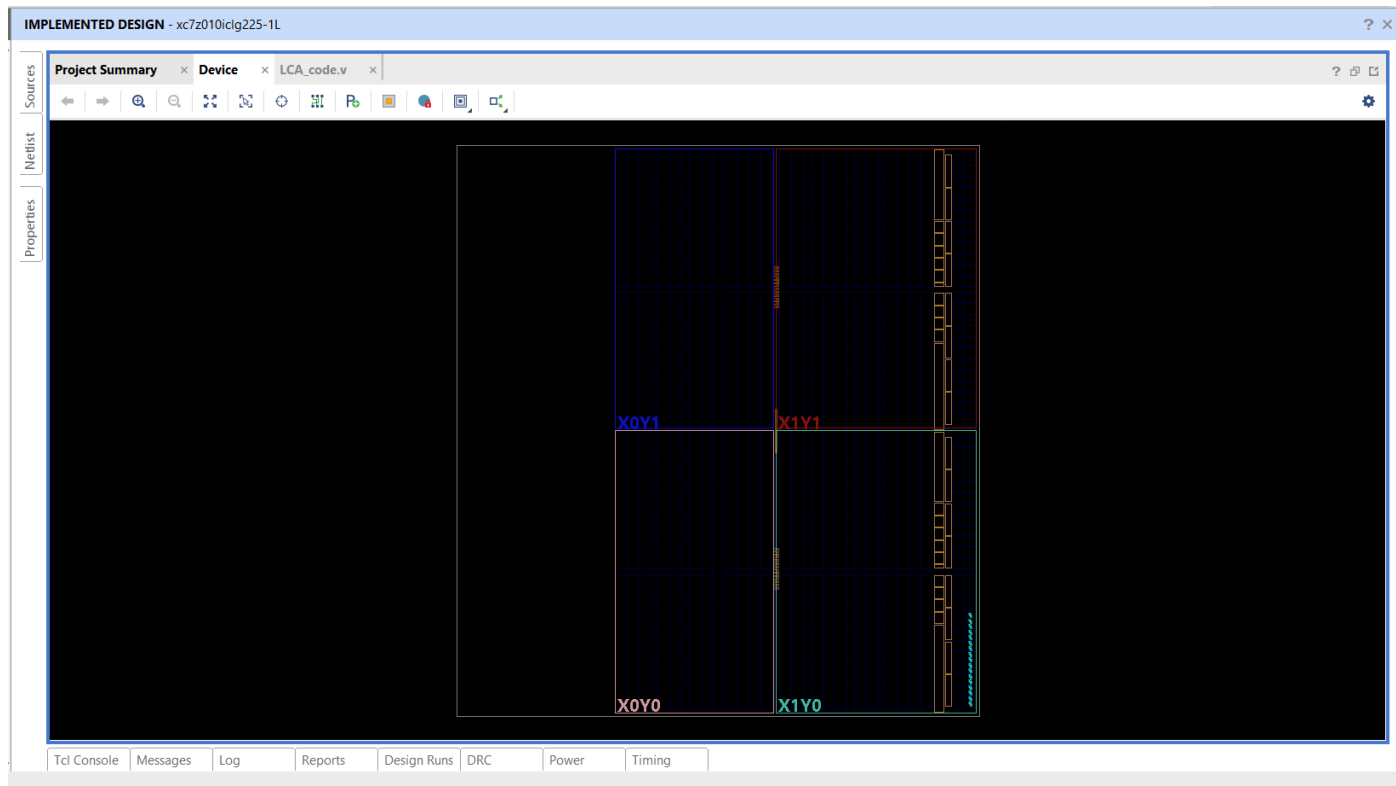
## 13. FPGA Device View

The implemented design device view showed successful placement and routing across the FPGA fabric.

The routed design confirmed:

- successful logic mapping
- proper implementation completion
- optimized hardware realization

**Figure 8: FPGA Implemented Device View**



## 14. Observations

The following observations were made during the project:

- Carry Lookahead Adders are significantly faster for larger bit-width operations.
- Parameterized Verilog design improves scalability and code reusability.
- Randomized testing improves functional verification coverage.
- FPGA implementation flow involves both logical verification and physical hardware mapping.
- Proper constraint files are necessary for successful bitstream generation.

## 15. Challenges Faced

Some challenges encountered during the project included:

- Understanding generalized carry-lookahead equations
- Implementing parameterized nested-loop carry logic
- Debugging synthesis and implementation warnings
- Understanding FPGA pin constraints and implementation flow

These challenges helped improve understanding of RTL design and FPGA workflows.

## 16. Learning outcomes

Through this project, the following concepts were learned:

- RTL design using Verilog HDL
- Parameterized hardware design
- Carry Lookahead Adder architecture
- Functional verification using testbenches
- Randomized verification techniques
- FPGA synthesis and implementation flow
- Timing and power analysis in Vivado
- Importance of FPGA constraints and pin mapping

## 17. Conclusion

This project successfully demonstrated the design and implementation of a parameterized Carry Lookahead Adder using Verilog HDL. The design was verified using both directed and randomized testing techniques. Functional simulation results confirmed correct operation for multiple bit-width configurations.

The project also provided practical exposure to FPGA synthesis, implementation, timing analysis, and power analysis using Vivado. The modular and parameterized architecture improved scalability and design flexibility.

Overall, the project strengthened understanding of digital arithmetic design, hardware description languages. It also provided foundational exposure to practical FPGA-oriented digital design methodologies and hardware verification workflows.

## 18. Future Scope

Future improvements may include:

- Hierarchical Carry Lookahead Adders
- FPGA hardware deployment using physical switches and LEDs
- SystemVerilog-based verification
- Timing optimization for larger bit-widths

## 19. References

- Verilog HDL Documentation
- Xilinx Vivado User Guide
- FPGA Architecture and Design References
- Carry Lookahead Adder Theory and Digital Logic Notes